

CO1

AT1

Name : K-NAVYA

Regno : 192371022

Course : CSA 0624

Data Analysis and Algorithms



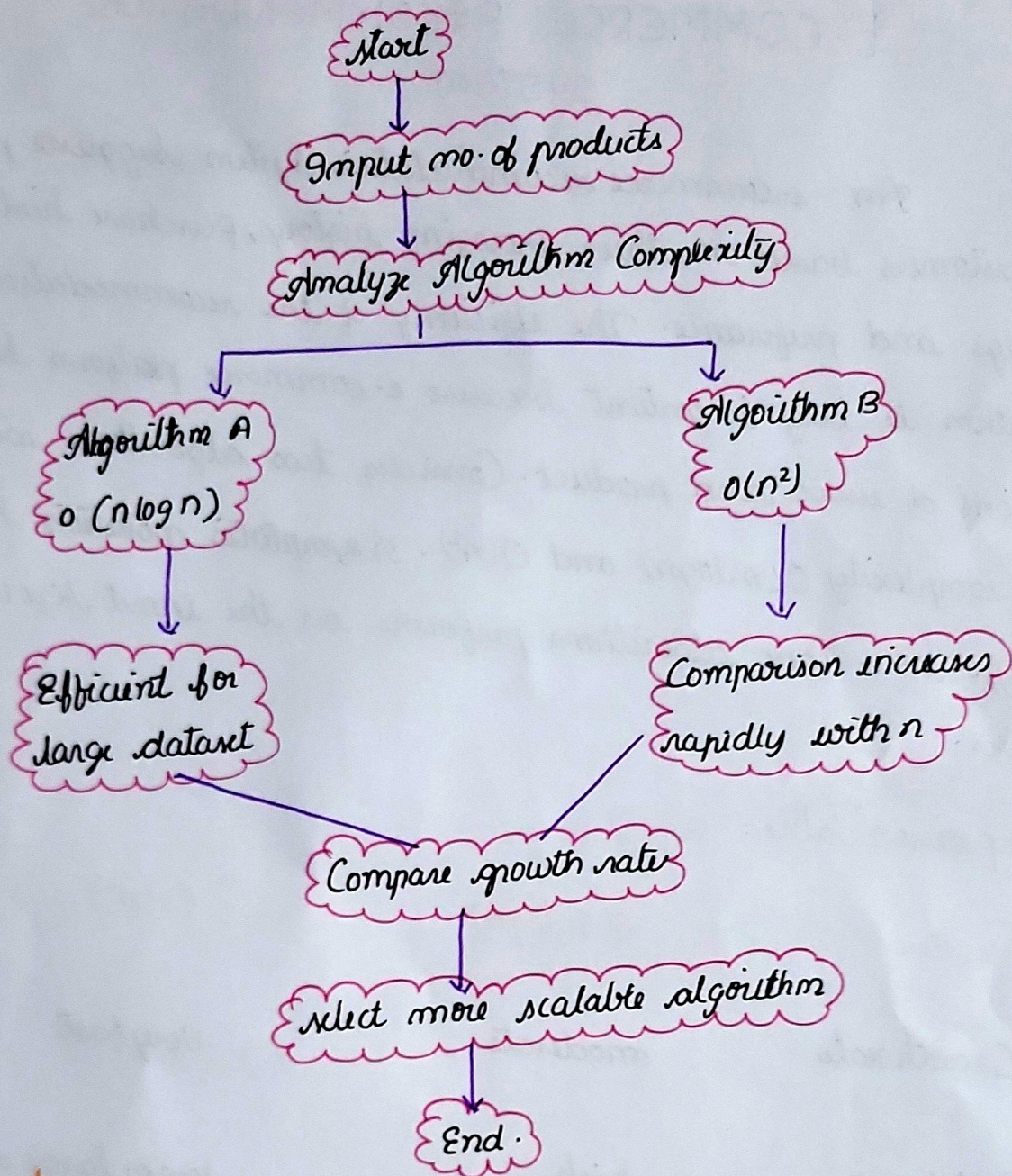
# E-COMMERCE RECOMMENDATION SYSTEM

An e-commerce recommendation system suggests product to customers based on their browsing history, purchase history, ratings and preference. The efficiency of the recommendation algorithm is very important because e-commerce platform handle millions of users and product. Consider two algorithms with time complexity  $O(n \log n)$  and  $O(n^2)$ . Asymptotic notation helps analyze how these algorithms performs as the input size  $n$  increases.

Comparison Table:

| Feature        | $O(n \log n)$  | $O(n^2)$           |
|----------------|----------------|--------------------|
| Growth rate    | moderate       | Very fast          |
| Efficiency     | high           | low for large data |
| Scalability    | Excellent      | Poor               |
| Suitable for   | Large datasets | Small datasets     |
| Execution time | Faster         | Slower             |





### Advantages:

- Faster execution for larger dataset
- Better scalability
- Reduced processing time

### Disadvantages:

- Poor scalability
- Longer response time



## CLOUD STORAGE PROCESSING SYSTEM

A cloud storage processing system manages and processes millions of files uploaded by users. The processing time is represented by recurrence relation:

$$T(n) = 3T(n/2) + n$$

where;

$n$  = number of files

$3T(n/2)$  = the problem is divided into 3 subproblems, each of size  $n/2$

$+n$  = Time required to combine

The master theorem is used to determine the asymptotic time complexity of the recurrence relation

The given recurrence relation is  $T(n) = 3T(n/2) + n$

$a = 3$  (number of recursive subproblems)

$b = 2$  (each subproblem size is divided by 2)

$f(n) = n$  (extra work outside recursion)

$$n \log^b a = n \log_2^3 \approx n^{1.585}$$

$$f(n) = n = O(n^{1.585 - \epsilon})$$



| Parameter  | Value | Meaning                     |
|------------|-------|-----------------------------|
| a          | 3     | no. of recursive problems   |
| b          | 2     | input size reduced by half  |
| f(n)       | n     | work done outside recursion |
| $\log_2 3$ | 1.585 | growth exponent             |

compute  $n^{\log_2 3} = n^{1.585}$

Compute  $f(n) = n$  with  $n^{1.585}$

Since  $n$  grows slower than  $n^{1.585}$ ,

$$T(n) = O(n^{1.585})$$

### Keywords :

uses master's theorem to solve recurrence solutions

parameter  $a=3, b=2, f(n)=n$

master theorem case 1 is applicable

Time complexity :  $O(n^{1.585})$

more efficient than  $O(n^2)$  for large dataset

provides good scalability in cloud storage processing.



## FINANCIAL ANALYSIS SYSTEM

Given the recurrence:

$$T(n) = 2T(n/2) + n \log n$$

Identify parameters:

$$a=2$$

$$b=2$$

$$f(n) = n \log n$$

Compute  $n \log_b a$

$$n \log^2 = n^1 = n$$

$$n \log_b^a = n$$

$$f(n) = n \log n = O(n \log n)$$

$$f(n) = O(n \log_b^a \log^k n), \quad k=1$$

According case 2:

$$T(n) = O(n \log_b^a \log^{k+1} n)$$

Substituting  $k=1$ :

$$T(n) = O(n \log^2 n)$$



## Performance Analysis for large Datasets:

- The algorithm divides the problem into two equal halves at each level.
- Each level performs  $n \log n$  work.
- There are  $\log n$  levels in the recursive tree
- Therefore, the work is:

$$n \log n \times \log n = n \log^2 n$$

## Interpretation Efficiency:

Time Complexity:  $O(n \log^2 n)$

- This is efficient for large financial datasets because it grows much slower than quadratic algorithms
- Although it is slightly slower than  $O(n \log n)$  due to the extra algorithmic factor
- it remains scalable and suitable for application such as financial analysis, Transaction processing.



# BANKING TRANSACTION PROCESSING - SYSTEM

Given the recurrence:

$$T(n) = 4T(n/2) + n^2$$

it using the master theorem

Identify the parameters:  $a = 4$

$$b = 2$$

$$f(n) = n^2$$

Compute  $n \log_b a$

$$n \log_2^4 = n^2$$

$$n \log_b^a = n^2$$

Since

$$f(n) = O(n \log_b^a \log^0 n)$$

$$f(n) = O(n \log_b^a \log^k n), k=0$$

According to case 2:

$$T(n) = O(n \log_b^a \log^{k+1} n)$$

$k=0$ :

$$T(n) = O(n^2 \log n)$$



## Performance Analysis for large transaction volumes

- The algorithm divides the transaction set into 4 subproblems each size of  $n/2$  size
- The work performed at each level is  $n^2$
- There are  $\log n$  levels of recursive.

Total work:

$$n^2 \times \log n = n^2 \log n$$

As the number of banking transaction, the running time grows as  $n^2 \log n$ . This is more expensive than  $O(n \log n)$  algorithm but more efficient than cubic time algorithm

Time Complexity =  $O(n^2 \log n)$

- The algorithm scales reasonably for moderate transaction volumes but becomes computationally expensive for very large datasets due to quadratic factor.
- Many banking transaction processing tasks, it is less efficient than algorithm for extremely large scale system.



# GAMING ENGINE OPTIMIZATION

A gaming engine must render thousands of objects every frame. Suppose the engine is evaluating three rendering algorithms

Algorithm A:  $O(n)$

Algorithm B:  $O(n \log n)$

Algorithm C:  $O(n^2)$

## Impact of Growth Rate:

$O(n)$ :

Growth is proportional to input

Doubling the number of object doubles the work

Suitable for real-time rendering

$O(n \log n)$ :

Growth is slightly faster than linear

Excellent for sorting objects before rendering

used in many modern graphics

$O(n^2)$ :

Growth is rapidly faster

Causes frame drops and lag in high resolution games.



